

**Universidad
Rey Juan Carlos**

Grado Inteligencia Artificial

Procesamiento del Lenguaje Natural I

PRACTICA 1

Realizado por:

- Raúl Vicente Sánchez
- Iván de Rada López

Índice

1. Introducción.
 - 1.1. Contexto proyecto y dataset TCGA.
 - 1.2. Definición del problema.
 - 1.3. Objetivos y metodología.
2. Propuestas, experimentos y resultados.
 - 2.1. Análisis exploratorio de datos.
 - 2.1.1. Distribución de la variable objetivo(T1-T4).
 - 2.1.2. Análisis de la longitud de los informes.
 - 2.2. Fase de Preprocesamiento Lingüístico.
 - 2.2.1. Preprocesamiento básico
 - 2.2.2. Lematización (spaCy)
 - 2.2.3. Filtrado POS (Part Of Speech)
 - 2.2.4. Lematización (NLTK)
 - 2.2.5. Stemming (NLTK)
 - 2.2.6. No preprocesamiento
 - 2.3. Experimentación
 - 2.3.1. Representaciones mediante vectores
 - 2.3.1.1. Pesado binario
 - 2.3.1.2. TF-IDF
 - 2.3.1.3. Frecuencia Absoluta (Bag-Of-Words)
 - 2.3.2. Empleo de modelos “clásicos”
 - 2.3.2.1. Naive Bayes
 - 2.3.2.2. Regresión Logística
 - 2.3.2.3. SVM (Support Vector Machine)
 - 2.3.3. Empleo de embeddings estáticos
 - 2.3.3.1. Word2Vec
 - 2.3.3.2. FastText
 - 2.3.3.3. GloVe
 - 2.3.3.4. Doc2Vec
 - 2.3.4. Empleo de embeddings contextuales
 - 2.3.4.1. BERT
 - 2.3.5. Empleo de modelos de Deep Learning
 - 2.3.5.1. CNN (Convolutional Neural Network)
 - 2.3.5.2. RNN (Recurrent Neural Network)
 - 2.3.5.2.1. LSTM (Long Short-Term Memory)
 - 2.3.5.2.2. Bi-LSTM (Bidirectional LSTM)
 - 2.3.5.3. Aumentado de datos
3. Conclusiones
4. Bibliografía y referencias

1. Introducción

1.1. Contexto proyecto y dataset TCGA.

Este trabajo se basa en el repositorio **The Cancer Genome Atlas (TCGA)**, una fuente global de datos clínicos y genómicos. Su mayor valor reside en los informes de patología, documentos en texto libre donde los especialistas detallan cada diagnóstico.

Sin embargo, la variabilidad del lenguaje médico hace que el análisis manual sea lento y propenso a errores. En este escenario, el Procesamiento de Lenguaje Natural (PLN) es la herramienta clave para transformar estos relatos en datos estructurados, permitiendo que la experiencia escrita del patólogo se convierta en una guía precisa para la toma de decisiones clínicas.

1.2. Definición del problema.

El objetivo central de este trabajo es la clasificación automática del estadio T (Tumor) dentro del sistema internacional TNM. El estadio T es un parámetro crítico en oncología, ya que describe el tamaño del tumor primario y el grado de invasión en los tejidos circundantes. En este corpus, nos enfrentamos a un problema de **clasificación multiclase**, donde cada informe debe ser etiquetado en uno de los cuatro niveles estándar:

- **T1:** Tumor limitado al órgano de origen, generalmente de tamaño reducido.
- **T2 y T3:** Indican un crecimiento progresivo en tamaño o profundidad de invasión.
- **T4:** El estadio más avanzado, donde el tumor ha invadido órganos vecinos o estructuras vitales.

La dificultad técnica radica en la naturaleza del lenguaje médico: un dominio saturado de terminología técnica, acrónimos ambiguos y, sobre todo, la presencia constante de negaciones y contextos complejos. El reto es lograr que un modelo computacional "entienda" estas sutilezas para asignar el estadio correcto basándose únicamente en la descripción narrativa del patólogo.

1.3. Objetivos y metodología.

El propósito fundamental de esta investigación es realizar una comparativa entre diferentes aproximaciones de procesamiento de texto para determinar cuál ofrece el mejor rendimiento en la predicción de estadios tumorales. Para ello, se han definido los siguientes objetivos específicos:

1. **Evaluación del Preprocesamiento:** Analizar cómo influyen la limpieza, la lematización (usando herramientas como spaCy y NLTK) y el filtrado por categorías gramaticales en la calidad de los datos de entrada.
2. **Representación Vectorial:** Comparar métodos clásicos basados en frecuencias, como TF-IDF, con representaciones más modernas basadas en Embeddings estáticos y contextuales.
3. **Modelado Predictivo:** Testear algoritmos tradicionales de aprendizaje automático (SVM, Regresión Logística) frente a arquitecturas de aprendizaje profundo (Redes Neuronales).

Para garantizar la validez de los resultados, se empleará la métrica Macro-F1 como indicador principal de éxito. Esta elección responde a la necesidad de evaluar el desempeño del modelo de manera equilibrada en todas las clases, evitando que el sesgo hacia las etiquetas más frecuentes oculte un mal rendimiento en los estadios menos representados, pero clínicamente críticos (como el T4).

2. Propuestas, experimentos y resultados.

2.1. Análisis exploratorio de datos.

En primer lugar, antes de realizar ningún análisis sobre los datos recibidos, tenemos que comprobar que todos los valores sean válidos, es decir que no haya ningún valor nulo en todo el dataset proporcionado, Esto puede llevarse a cabo mediante:

- `print(df.isnull().sum())` – Lo cual calcula la suma de valores nulos del DataFrame (*Figura 1*).

```
patient_id    0
text          0
t             0
dtype: int64
```

Figura 1

- `df.info()` – Que nos muestra información sobre el DataFrame incluido si hay algún valor nulo en alguna de las columnas (*Figura 2*).

```
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   patient_id  5158 non-null     str
1   text        5158 non-null     str
2   t           5158 non-null     str
dtypes: str(3)
```

Figura 2

También realizamos una comprobación de si hay algún dato duplicado lo cual nos podría perjudicar a la hora de futuros cálculos.

2.1.1. Distribución de la variable objetivo (T1-T4).

El primer paso consiste en analizar la frecuencia de cada estadio tumoral. Se observa un **desequilibrio de clases**: algunas categorías (como T2 o T3) suelen tener una presencia mucho mayor que los casos extremos (T1 o T4). Este desajuste es crítico, ya que condicionará la elección de la métrica **Macro-F1** para asegurar que el modelo aprenda a identificar correctamente los estadios menos frecuentes y no solo los mayoritarios.

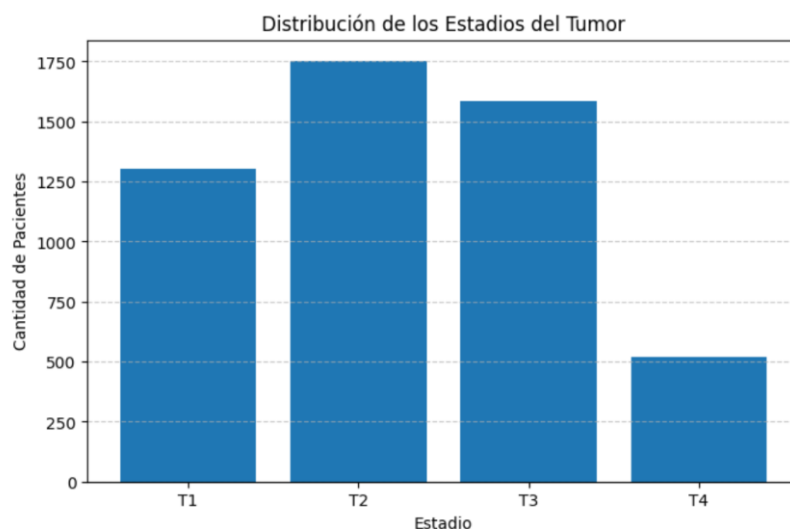


Figura 3

2.1.2. Análisis de la longitud de los informes.

Para profundizar en la naturaleza del corpus, se ha analizado la extensión de los textos mediante un diagrama de caja (boxplot). Esta visualización es fundamental, ya que no solo muestra la longitud media de los informes, sino que permite comparar la distribución del volumen de palabras entre los cuatro estadios tumorales.

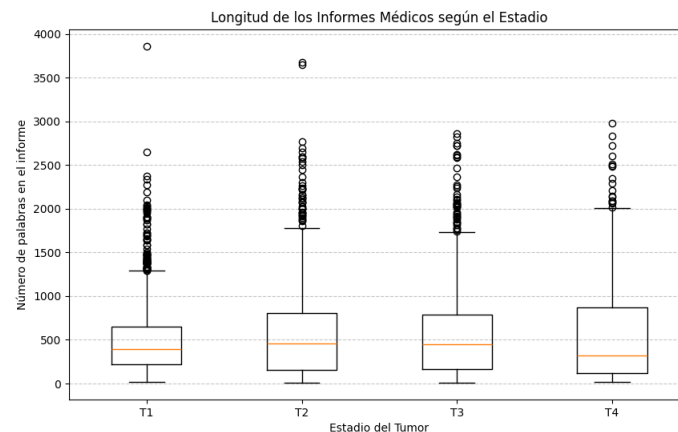


Figura 4

Los resultados revelan la presencia de **valores atípicos** (outliers), correspondientes a informes excepcionalmente extensos o breves. La observación de las medianas y los cuartiles sugiere si existe una correlación entre la complejidad del estadio (por ejemplo, un T4 que invade más tejidos) y la longitud de su descripción. Este análisis justifica la necesidad de normalizar los textos para que la extensión del informe no se convierta en un sesgo que confunda a los modelos de clasificación (*Figura 4*).

2.2. Fase de preprocesamiento lingüístico

El texto libre médico contiene mucho "ruido" que no aporta valor predictivo. Se diseñó un torneo de preprocesamiento probando cinco estrategias distintas para encontrar la representación más limpia que será la que usemos posteriormente. Se han usado dos librerías diferentes para comparar resultados entre ellas.

2.2.1. Preprocesamiento básico

En primer lugar, realizamos un preprocesamiento básico mediante la librería de spaCy. Empleamos esta librería inicialmente debido a que esta altamente optimizada para poder procesar grandes volúmenes de textos a una velocidad mayor.

Esta función de preprocesamiento consiste en convertir todas las letras a minúsculas para también poder filtrarlas mediante expresiones regulares empleadas para poder eliminar del texto todo aquello que no sean palabras o letras presentes en el alfabeto, así también los signos de puntuación. Posteriormente se eliminan las **stop words**^[1] y se guardan los tokens resultantes en una lista. (Figura 5)

```
def preprocesamiento_basico(texto):
    texto = str(texto).lower()
    texto = re.sub(r'^a-z\s', ' ', texto)
    texto = re.sub(r'\s+', ' ', texto).strip()

    doc = nlp(texto)
    tokens_limpios = [token.text for token in doc if not token.is_stop]

    return " ".join(tokens_limpios)
```

Figura 5

[1] Stop Words: Palabras muy comunes y frecuentes en un idioma que, por lo general, se filtran y eliminan antes o después de procesar datos de texto en el ámbito del Procesamiento del Lenguaje Natural (PLN) y los motores de búsqueda.

2.2.2. Preprocesamiento Lematización (spaCy)

En esta función realizamos lo mismo que anteriormente, pero emplearemos un filtro más que consiste en realizar la lematización^[2] de los diferentes tokens y guardarlos en una lista como anteriormente. Cabe resaltar que las Expresiones Regulares son importantes ya que nos evita diversos problemas:

- Evita problemas de reconocimiento de palabras, ya que eliminamos las comas, los paréntesis...
- Ahorramos procesamientos innecesarios como puede ser intentar lematizar números.

```
def preprocesamiento_lematizacion(texto):  
    texto = str(texto).lower()  
    texto = re.sub(r'^a-z\s', ' ', texto)  
    texto = re.sub(r'\s+', ' ', texto).strip()  
  
    doc = nlp(texto)  
    tokens_lemma = [token.lemma_ for token in doc if not token.is_stop]  
  
    return " ".join(tokens_lemma)
```

Figura 6

[2] Lematización: Proceso fundamental en la lingüística y el Procesamiento de Lenguaje Natural (PLN) que consiste en reducir una palabra a su forma base o de diccionario, la cual se conoce como **lema**.

2.2.3. Filtrado POS (Part Of Speech)

La función (Figura 7) implementa un *pipeline* de procesamiento estricto diseñado para extraer exclusivamente la carga semántica más relevante del texto clínico. Su ejecución consta de tres fases fundamentales:

1. **Normalización y limpieza:** Convierte el texto a minúsculas y utiliza expresiones regulares para eliminar números y signos de puntuación, garantizando una entrada estrictamente alfabética.
2. **Análisis morfosintáctico:** Procesa el texto limpio mediante el motor estadístico de spaCy (en_core_web_sm) para extraer las etiquetas gramaticales (*Part of Speech*) y el lema de cada palabra en base a su contexto.
3. **Filtrado semántico y lematización:** Aplica una heurística de selección estricta que descarta las palabras vacías (*stop words*) y conserva únicamente **sustantivos, nombres propios y adjetivos** (NOUN, PROP, ADJ).

Al extraer únicamente los lemas de estas tres categorías, el sistema elimina el ruido sintáctico (como verbos o adverbios) y genera una representación textual densa, enfocada exclusivamente en las entidades anatómicas y sus descriptores patológicos.


```
def preprocesamiento_filtrado_pos(texto):
    texto = str(texto).lower()
    texto = re.sub(r'^a-z\s', ' ', texto)
    texto = re.sub(r'\s+', ' ', texto).strip()

    doc = nlp(texto)
    categorias_permitidas = ['NOUN', 'PROPN', 'ADJ']

    tokens_filtrados = [
        token.lemma_ for token in doc
        if not token.is_stop and token.pos_ in categorias_permitidas
    ]

    return " ".join(tokens_filtrados)
```

Figura 7

2.2.4. Lematización (NLTK)

Esta función (Figura 8) implementa un *pipeline* de normalización textual utilizando la librería **NLTK** (Natural Language Toolkit). Su ejecución se divide en tres fases principales:

1. **Limpieza troncal:** Convierte el texto a minúsculas y utiliza expresiones regulares (`re.sub`) para aislar exclusivamente los caracteres alfabéticos, eliminando el ruido introducido por números, símbolos y signos de puntuación.
2. **Tokenización:** Emplea el motor estadístico `word_tokenize` de NLTK para dividir la cadena de texto continua en unidades individuales (palabras o *tokens*).
3. **Filtrado y Lematización conjunta:** Mediante una lista por comprensión iterativa, el algoritmo evalúa cada *token*. Primero descarta aquellos que coinciden con las palabras vacías (*stop words*) del diccionario en inglés de NLTK. A los términos que superan este filtro se les aplica el algoritmo de lematización (`lemmatizer.lemmatize`), reduciéndolos a su forma base de diccionario.

El objetivo de esta función es generar una representación del texto estandarizada morfológicamente y libre de ruido. Además, su implementación está justificada metodológicamente para poder contrastar el rendimiento del motor léxico de NLTK (basado en el diccionario WordNet) frente a la lematización contextual de spaCy en el dominio clínico.

```
def preprocesamiento_lematizacion_nltk(texto):
    texto = str(texto).lower()
    texto = re.sub(r'[^\w\s]', ' ', texto)
    texto = re.sub(r'\s+', ' ', texto).strip()

    # 1. Tokenizamos usando NLTK
    tokens = nltk.word_tokenize(texto)

    # 2. Lematizamos y filtramos las stop words en el mismo paso
    tokens_lemma = [
        lemmatizer.lemmatize(palabra)
        for palabra in tokens
        if palabra not in stop_words_nltk
    ]

    return " ".join(tokens_lemma)
```

Figura 8

2.2.5. Stemming (NLTK)

Implementa una estrategia de normalización textual (*Figura 9*) mediante el uso de la librería **NLTK**. Su diseño se estructura en tres etapas clave:

1. **Limpieza troncal:** Estandariza el texto a minúsculas y aplica expresiones regulares (`re.sub`) para preservar únicamente los caracteres alfabéticos, suprimiendo de forma robusta signos de puntuación, números y espacios redundantes.
2. **Tokenización básica:** Divide la cadena de texto continua en unidades individuales (*tokens*) utilizando la función nativa de separación por espacios (`.split()`), una alternativa más rápida que los tokenizadores estadísticos completos.
3. **Filtrado y Stemming:** A través de una lista por comprensión iterativa, el algoritmo descarta primero las palabras vacías (*stop words*) del diccionario de NLTK. Seguidamente, a los términos relevantes se les aplica el algoritmo **PorterStemmer** (`stemmer.stem`), el cual trunca las palabras hasta su raíz morfológica (lexema) mediante reglas empíricas de eliminación de sufijos.

A diferencia de la lematización estricta, el *Stemming* no busca validar la palabra en un diccionario oficial, sino que agrupa de forma agresiva múltiples variaciones de una misma familia léxica bajo un único *token* matemático (por ejemplo, reduciendo "cancerous" a "cancer"). En la fase experimental de este estudio, esta técnica demostró ser la más eficiente para concentrar la señal estadística del vocabulario médico y maximizar la precisión de los modelos de *Machine Learning* clásicos.

```
def preprocesamiento_stemming_basico(texto):  
  
    texto = str(texto).lower()  
    texto = re.sub(r'^a-z\s', ' ', texto)  
    texto = re.sub(r'\s+', ' ', texto).strip()  
  
    # Tokenizamos separando por espacios  
    tokens = texto.split()  
  
    # Aplicamos filtro de stopwords y luego el Stemmer  
    tokens_stemmed = [  
        stemmer.stem(palabra)  
        for palabra in tokens  
        if palabra not in stop_words_nltk  
    ]  
  
    return " ".join(tokens_stemmed)
```

Figura 9

2.2.6. No preprocesamiento

Como se puede observar también hemos tenido en cuenta la opción de no realizar ningún preprocesamiento al texto, esto se debe principalmente a un detalle importante presente en todos los demás preprocesamientos:

- **Preservación de marcadores clínicos numéricos:** Las técnicas empleadas anteriormente estaban en parte basadas en expresiones regulares, de esta forma eliminábamos los números y los caracteres especiales. Sin embargo, en el dominio oncológico, las variables críticas para determinar la variable objetivo (T1 – T4) son mayormente alfanuméricas (p.e, “*tamaño tumoral de 4.5 cm*”). El texto sin preprocesar garantiza la supervivencia de estos biomarcadores en el conjunto de entrenamiento.
- **Adecuación futuras arquitecturas secuenciales:** Aunque los modelos clásicos (Sección 2.3.2) basados en frecuencias matemáticas (TF-IDF) se benefician de la agrupación de vocabulario mediante *Stemming*, las arquitecturas basadas en *Embeddings* (Sección 2.3.3) y Redes Neuronales operan bajo otro paradigma. Estos algoritmos necesitan modelar ventanas de contexto continuo, para lo cual la presencia de palabras vacías (*stop words*), la sintaxis intacta y las partículas de negación clínica son fundamentales para no alterar el sentido de la frase.

En conclusión, esta columna de datos también se someterá a evaluación en las diferentes fases junto con el resto de las propuestas preprocesadas, partiendo de la hipótesis de que ciertas arquitecturas avanzadas podrían extraer características más ricas de la sintaxis natural que de un texto fragmentado por reglas lingüísticas deterministas.

2.3 Experimentación

2.3.1 Representaciones mediante vectores

Para que los modelos de Machine Learning puedan procesar y aprender del corpus, es fundamental transformar el texto una vez preprocesado en un formato numerico que la maquina sea capaz de entender, a esto se le llama vectorización.

Nosotros para ello hemos usado dos tecnica diferentes, el pesado binario y TF-IDF.

2.3.1.1. Pesado binario

Este método consiste en crear una bolsa de palabras en la que se encuentren todas las palabras del corpus, una vez ya creada, se compara cada informe médico con dicha bolsa creando un vector que contiene 0 y 1 del tamaño de la bolsa de palabras, donde el 0 representa que ese término no se encuentra en el informe y 1 representa que sí.

Esta técnica es de gran relevancia en nuestro proyecto, ya que, permite detectar la existencia de un término como puede ser “metástasis” que puede ser un gran indicador del estadio del tumor, independientemente de cuantas veces se haya repetido la palabra en el informe. El pesado binario evita que lo términos más repetidos dominen el vocabulario

2.3.1.2. TF-IDF

A diferencia del pesado binario, el TF-IDF permite ponderar la relevancia de cada término en un documento especifico respecto a toda la colección de historiales médicos.

Esta técnica se compone de dos partes:

1. Term Frecuency: Mide la frecuencia con la que aparece un término en un documento asumiendo que, si se repite mucho en ese diagnóstico, es muy significativo para ese diagnóstico.
2. Inverse Document Frecuency: Penaliza los términos que aparecen en la mayoría de los informes médicos del corpus.

La aplicación de esta tecnica en este proyecto es fundamental ya que, palabras como “paciente” o “tejido” aparece en casi todos los informes, haciendo que su relevancia

para el estadio sea prácticamente nula. Por lo que el componente IDF las penaliza y eleva el peso de palabras más específicas y relevantes.

2.3.1.3 Frecuencia Absoluta (Bag-Of-Words)

A diferencia del pesado binario (que solo indica si un término está presente o no), la técnica de Frecuencia Absoluta (*Bag of Words*) cuenta exactamente cuántas veces se repite cada palabra dentro de un informe médico.

Para este experimento, configuramos el modelo para trabajar solo con las 5.000 palabras más comunes (`"max_features=5000"`), manteniendo así la misma dimensionalidad que en las pruebas anteriores. La hipótesis para probar esta técnica era muy intuitiva: si un patólogo repite muchos conceptos críticos como *"infiltración"* o *"invasión"*, podíamos asumir que esto indicaría una mayor gravedad en el tumor (estadios T3 o T4).

Sin embargo, este enfoque tiene una limitación importante. Al no aplicar ninguna penalización global a los términos repetitivos (algo que sí soluciona el método TF-IDF), corremos el riesgo de que las palabras médicas genéricas y muy comunes terminen teniendo más peso para el algoritmo que los verdaderos biomarcadores que definen el estadio del cáncer.

2.3.2 Modelos clásicos

Tras la fase de vectorización se ha procedido con la implementación de modelos de aprendizaje supervisado clásicos debido a su capacidad de interpretación en modelos médicos y para establecer un baseline robusto.

Debido a que es un problema de clasificación multiclase en cuatro categorías (T1, T2, T3, T4) hemos decidido implementar tres algoritmos concretos.

2.3.2.1. Naive Bayes

Al evaluar Naive Bayes con nuestras tres técnicas de vectorización, los resultados han dejado claro que ha sido el algoritmo con el rendimiento más bajo de todo el bloque de modelos clásicos.

Sus mejores puntuaciones en la métrica Macro-F1 fueron:

- Pesado Binario (con texto crudo): 0.5172.

- Frecuencia Absoluta (con filtrado POS): 0.5127.
- TF-IDF (con texto básico): 0.4655.

El problema principal de este algoritmo radica en su propia base teórica. Recibe el nombre de "ingenuo" (*naive*) porque asume que la aparición de una palabra en el texto es completamente independiente de las demás. Sin embargo, en la redacción médica, esto es totalmente falso.

Si en un informe aparece la palabra "carcinoma", es altísimamente probable que la acompañen términos como "infiltrante" o "invasión". Al ignorar por completo estas relaciones lógicas y tratar cada palabra como un evento aislado, el modelo pierde el sentido real de la frase. Esta falta de contexto le ha impedido clasificar correctamente los casos más ambiguos y los estadios minoritarios (T4), dejándolo muy por detrás de otros modelos lineales.

2.3.2.2. Regresión Logística

La Regresión Logística multinomial ha sido la gran sorpresa de los modelos clásicos, demostrando un rendimiento excelente para clasificar los estadios tumorales.

Sus mejores resultados de Macro-F1 (siempre con el texto crudo) fueron:

- Pesado Binario: 0.7040.
- Frecuencia Absoluta: 0.6741.
- TF-IDF: 0.6211.

La razón por la que este modelo ha alcanzado un gran 0.7040 con el pesado binario es muy interesante. Al algoritmo se le da genial manejar matrices dispersas con muchas columnas. En los informes médicos, lo que de verdad le importa al modelo es saber si una palabra clave crucial (como "*metástasis*", "*infiltración*" o "*carcinoma*") aparece o no en el texto. El pesado binario le da un "1" si existe, y la Regresión Logística le asigna un peso matemático muy alto y positivo a esa palabra. Así, cuando el modelo lee esa palabra, sabe casi automáticamente que el paciente está en un estadio avanzado sin necesidad de contar cuántas veces se repite.

Además, al usar la columna de texto crudo (text), el modelo ha podido aprovechar los números y códigos médicos que los preprocesamientos más agresivos borran. Como en oncología el tamaño del tumor en centímetros (por ejemplo, "*4.5 cm*") es vital para decidir si es un estadio T1 o T3, mantener estos datos numéricos intactos le ha dado a la Regresión Logística las pistas exactas que necesitaba para acertar en la clasificación.

2.3.2.3. SVM (Support Vector Machine)

El algoritmo SVM se ha consolidado como el segundo mejor modelo clásico de nuestro estudio, logrando un desempeño muy robusto y consistente. Al igual que ocurrió con el resto de los modelos lineales, su pico de rendimiento se alcanzó utilizando la columna de texto crudo (text), obteniendo los siguientes resultados en Macro-F1:

- TF-IDF: 0.6848.
- Pesado Binario: 0.6624.
- Frecuencia Absoluta: 0.6207.

Como comenté anteriormente en la memoria, las SVM son excelentes trazando fronteras de separación en espacios muy complejos y con muchas dimensiones, fijándose precisamente en los informes médicos más ambiguos (los vectores de soporte). Esto es fundamental en oncología, donde la diferencia entre un tumor T2 y un T3 a veces depende de una sola frase o de un milímetro de tamaño. Al pasarle el texto sin preprocesar, la SVM pudo aprovechar todas esas medidas numéricas y códigos originales para dibujar un hiperplano de separación mucho más preciso.

El éxito con TF-IDF y el problema de convergencia a diferencia de la Regresión Logística (que prefirió el Pesado Binario), la SVM logró su mejor puntuación trabajando en equipo con la vectorización TF-IDF. Esto tiene todo el sentido matemático: como las SVM calculan márgenes y distancias geométricas, necesitan que los datos estén bien escalados. El TF-IDF normaliza los pesos de las palabras, penalizando las muy comunes y destacando las raras.

Por el contrario, un detalle técnico muy relevante que observamos en la consola de ejecución fue que **la SVM no lograba converger al usar la Frecuencia Absoluta** (*“Liblinear failed to converge”*). Al pasarle números absolutos muy grandes y dispares (porque algunas palabras se repiten decenas de veces y otras solo una), al algoritmo le cuesta muchísimo más encontrar la frontera de separación óptima. Este error empírico nos confirma que, para modelos basados en distancias como SVM, vectorizar con TF-IDF o valores binarios es la opción correcta.

2.3.3. Empleo de embeddings estáticos.

A pesar de la interpretabilidad de los modelos clásicos basados en frecuencias (como *Bag of Words* o TF-IDF), estas representaciones presentan una limitación crítica para el Procesamiento de Lenguaje Natural avanzado: tratan cada palabra como un ente matemático aislado, ignorando por completo el contexto y las relaciones de sinonimia.

Para superar esta barrera, en esta fase se introducen los Embeddings Estáticos. Su implementación se justifica por su capacidad para capturar la semántica latente del

lenguaje oncológico mediante vectores continuos y densos. A diferencia del simple conteo de términos, los *embeddings* proyectan el vocabulario en un espacio geométrico donde términos con significados similares (ej. "tumor", "carcinoma", "neoplasia") se agrupan matemáticamente. Esta transición reduce drásticamente la dispersión de los datos y dota a los algoritmos de una comprensión real del contexto clínico, mejorando su capacidad de generalización frente a la variabilidad en la redacción de los patólogos.

2.3.3.1. Word2Vec

Word2Vec es una red neuronal de dos capas diseñada para reconstruir contextos lingüísticos de palabras. En lugar de representar cada palabra como un índice aislado en un vocabulario (como ocurre en TF-IDF), Word2Vec sitúa cada término en un espacio vectorial de dimensiones reducidas (en este estudio, $d=100$).

El principio subyacente es la Hipótesis Distribucional: *"palabras que aparecen en contextos similares tienden a tener significados similares"*. Mediante el entrenamiento, el modelo aprende a asignar coordenadas a las palabras de forma que términos oncológicos relacionados (por ejemplo, "metástasis", "infiltración" y "maligno") queden geométricamente cerca en el espacio latente.

El uso de este embedding estático se justifica por diferentes motivos:

- Captura de matices clínicos: A diferencia de los modelos de frecuencias, Word2Vec permite al modelo entender conceptos semánticamente cercanos, aunque sean palabras distintas. Esto dota al clasificador de una mayor robustez frente a las variaciones en la redacción de los informes.
- Densidad de la información: Mientras que las matrices TF-IDF suelen tener miles de columnas (una por palabra), Word2Vec comprime toda la información en 100 dimensiones densas.
- Entrenamiento ***From Scratch***: A diferencia de usar modelos pre-entrenados en Google News o Wikipedia, en este código se ha optado por entrenar el modelo exclusivamente con el corpus TCGA. Esto asegura que los vectores aprendan el lenguaje técnico específico de la patología oncológica, donde el contexto de una palabra puede diferir radicalmente del lenguaje cotidiano.

Dado que Word2Vec genera un vector por cada palabra y nuestra variable objetivo (T1-T4) se asigna al informe completo, se ha implementado una técnica de Promediado de

Vectores (Mean Pooling) (*Figura 10*). Cada historial médico se representa mediante el centroide de los vectores de sus palabras constituyentes:

$$V_{doc} = \frac{1}{n} \sum_{i=1}^n v_i$$

Donde n es el número de palabras conocidas por el modelo en el informe y v_i es el embedding de cada palabra. Este vector resultante actúa como la "huella digital semántica" del informe clínico.

```
def document_vector(doc_tokens, model):
    palabras_conocidas = [word for word in doc_tokens if word in model.wv.key_to_index]
    if len(palabras_conocidas) == 0:
        return np.zeros(model.vector_size)
    return np.mean(model.wv[palabras_conocidas], axis=0)
```

Figura 10

También cabe mencionar que tenemos varios detalles en el código que a continuación se explicaran (algunos de ellos modificados por los experimentos realizados):

- “*min_count = 2*”: Lo que realizamos con esto es ignorar palabras que solamente están presentes una vez en el informe una vez para poder eliminar el ruido presente, pero tras varias pruebas nos dimos cuenta de que, al eliminar esta variable de la declaración de nuestro modelo, nuestro “*macro-f1*” mejoraba ligeramente.
- “*window = 5*”: Inicialmente teníamos puesta una ventana de 5 palabras, es decir, que mira 5 palabras desplazándonos hacia la izquierda y de la misma forma a la derecha, pero al cambiar este valor a 20, observamos que también mejora la puntuación ligeramente, debe deberse a que entonces cada palabra obtiene más contexto por lo tanto en el momento de realizar las predicciones, son más acertadas.

Estrategia preprocesamiento	Macro-F1 (Regresión Logística)
Text_stem_nltk	0.5121
Text_lemma	0.5046
Text_pos	0.4998
Text_lemma_nltk	0.4966
Text_basico	0.4953
text	0.4677

Tras evaluar el modelo Word2Vec entrenado desde cero con las distintas variantes del texto, los resultados en Macro-F1 fueron los siguientes (de mejor a peor): Stemming (0.5121), Lematización spaCy (0.5046), POS (0.4998), Lematización NLTK (0.4966), Básico (0.4953) y Texto Crudo (0.4677).

De esta distribución se extraen tres conclusiones fundamentales:

1. El éxito del Stemming: Al entrenar el modelo desde cero (*from scratch*), el *Stemming* obtiene el mejor rendimiento. Al reducir múltiples variaciones de una palabra médica (ej. "*metastatic*", "*metastases*") a una única raíz ("*metastasi*"), se agrupa su frecuencia. Esto proporciona al modelo muchísimos más ejemplos de contexto para una misma idea, generando vectores matemáticamente muy sólidos.
2. El fracaso del Texto Crudo: Quedó en último lugar debido a la alta dispersión. Al no limpiar mayúsculas, signos de puntuación o variaciones, Word2Vec interpreta "*Tumor*", "*tumor*" y "*tumor,*" como palabras distintas. Esta fragmentación impide que el algoritmo acumule la estadística necesaria para aprender sus contextos.

A diferencia de los modelos pre-entrenados (que colapsan ante el *Stemming*), cuando se entrena un espacio vectorial local en un corpus de tamaño moderado, reducir agresivamente el vocabulario (mediante Stemming o Lematización) es la mejor estrategia para evitar la dilución semántica de los datos clínicos.

2.3.3.2. FastText

Tras la realización del modelo de Word2Vec, se implementó una arquitectura FastText. Esta arquitectura, pese a la similitud, introduce una mejora arquitectónica bastante relevante, la representación de caracteres basadas en los n-gramas.

A diferencia del Word2Vec que si por ejemplo encuentra una palabra con algún error ortográfico, le asigna un vector nulo, en FastText se reconoce gran parte de sus n-gramas aprendidos previamente infiriendo un vector matemático muy cercano al original.

```
ft_model = FastText(sentences=X_train_tokens, vector_size=100, window=20, min_count=2, workers=6, seed=42)
```

Figura X

Se explicarán algunos detalles procedentes del código (*Figura X*), principalmente parámetros con los que se instancia el modelo a entrenar:

- "*vector_size=100*": Compresión del texto en vectores densos de 100 dimensiones.
- "*window=20*": Como se ha mencionado en el anterior modelo (Word2Vec) este parámetro ha cambiado, anteriormente también era de 5 y tras una serie de experimentos y comparación de resultados, esta es la decisión final.

Al igual que en las aproximaciones previas, se empleó la técnica de Promediado de Vectores (*Mean Pooling*) (*Figura X*). Al calcular el centroide de los vectores inferidos por FastText para cada historial médico, se obtuvo una representación global densa que

posteriormente fue introducida en el clasificador de Regresión Logística para predecir el estadio tumoral.

Estrategia preprocesamiento	Macro-F1 (Regresión Logística)
text_lemma_nltk	0.5139
text_stem_nltk	0.4954
text_basico	0.4951
text_lemma	0.4934
text	0.4822
text_pos	0.4763

Tras evaluar el espacio vectorial de FastText, los resultados en Macro-F1 fueron (de mejor a peor): **Lematización NLTK** (0.5139), **Stemming** (0.4954), **Básico** (0.4951), **Lematización spaCy** (0.4934), **Texto Crudo** (0.4822) y **POS** (0.4763).

De esta distribución destacan tres conclusiones clave:

1. **Sinergia perfecta con la Lematización y el Stemming:** A diferencia de modelos pre-entrenados como GloVe que se rompen al leer palabras truncadas, FastText brilla aquí. Al leer "trozos de palabras" (n-gramas), agrupa perfectamente las raíces médicas truncadas por el Stemming o normalizadas por la lematización, creando vectores fortísimos sin perder información.
2. **Sensibilidad al ruido y a la sintaxis:** El texto crudo (text) penaliza al modelo porque la puntuación y las mayúsculas aleatorias alteran las distancias matemáticas entre palabras. Por su parte, el filtrado POS (text_pos) quedó último porque, al eliminar verbos y conectores, destruye la secuencia lógica de la frase que FastText necesita para aprender el contexto local.

La arquitectura sub-léxica de FastText demuestra ser la red de seguridad perfecta para textos médicos. Cuando se combina con técnicas de reducción de vocabulario (Lematización NLTK o Stemming), se obtiene la representación más robusta frente a la variabilidad clínica.

2.3.3.3. GloVe

La justificación del empleo de este modelo recae en que a diferencia de Word2Vec o FastText, en los que sus entrenamientos se basan en ventanas de contexto local, GloVe fundamenta su aprendizaje en la factorización de una matriz global de co-ocurrencias de palabras a lo largo de todo el corpus.

```
glove_model = api.load("glove-wiki-gigaword-100")
```

Figura 11

En lugar de entrenar el algoritmo desde cero exclusivamente con el corpus oncológico de TCGA, se procedió a cargar el modelo pre-entrenado “*glove-wiki-gigaword-100*” (Figura 11). Este modelo ha sido optimizado previamente sobre un corpus masivo de 6 billones de tokens procedentes de artículos de la Wikipedia y noticias.

- Al calcular el vector promedio del documento clínico (“*document_vector_glove*”), se aplicó un filtro estricto de existencia léxica (“*word in model.key_to_index*”). Esta es la mayor vulnerabilidad de utilizar modelos pre-entrenados frente a técnicas sub-léxicas como FastText. Si el modelo se enfrenta a la columna de texto con reducción morfológica (por ejemplo, *Stemming*, donde la palabra “*infiltrating*” se reduce a “*infiltrat*”), GloVe no reconocerá el término, ya que no existe en el diccionario normativo de la Wikipedia. En estos casos, la función retorna un vector nulo (*np.zeros*), lo que irremediabilmente diluye la señal predictiva del historial médico y penaliza el rendimiento del clasificador.
- Al igual que en los torneos previos, los vectores latentes generados por GloVe se introdujeron en un modelo de Regresión Logística, manteniendo un control estricto sobre la aleatoriedad (“*random_state=42*”) y la estratificación del corpus de prueba.

Estrategia preprocesamiento	Macro-F1 (Regresión logística)
text_lemma	0.4902
text_lemma_nltk	0.4831
text_pos	0.4827
text_basico	0.4763
text_stem_nltk	0.4714
text	0.4268

Los resultados obtenidos corroboraron las hipótesis iniciales sobre la vulnerabilidad de los modelos pre-entrenados ante el preprocesamiento destructivo y el lenguaje de dominio específico. El mayor rendimiento predictivo se alcanzó con las columnas lematizadas (text_lemma: 0.4902 Macro-F1), dado que esta técnica normaliza el corpus a formas de diccionario, maximizando la coincidencia (*match*) con el vocabulario normativo de la Wikipedia con el que GloVe fue entrenado. Por el contrario, la técnica de Stemming degradó severamente el rendimiento (0.4714) al generar raíces artificiales (Out-Of-Vocabulary) que el modelo global fue incapaz de vectorizar. De igual forma, la representación en texto crudo (text: 0.4268) sufrió una caída crítica, evidenciando que el ruido sintáctico inherente a un historial clínico (abreviaturas, códigos patológicos y erratas) resulta incomprensible para un modelo entrenado exclusivamente con corpus lingüísticos de dominio general.

2.3.3.4. Doc2Vec

Como última iteración dentro del bloque de *embeddings* estáticos, se implementó la arquitectura **Doc2Vec** (también conocida en la literatura como *Paragraph Vectors*). Mientras que Word2Vec, FastText y GloVe son modelos diseñados a nivel léxico (obligando a recurrir al promediado matemático o *Mean Pooling* para representar un informe completo), Doc2Vec extiende formalmente estas arquitecturas para codificar secuencias de texto de longitud variable en un único vector denso continuo.

La decisión de incorporar Doc2Vec en el marco experimental responde a la necesidad de evaluar un enfoque de representación global frente a las técnicas de agregación heurística. En un historial clínico oncológico, el promediado simple de vectores presenta el riesgo latente de diluir la señal de palabras clave críticas y poco frecuentes en un mar de términos médicos comunes.

Doc2Vec solventa esta limitación introduciendo un vector adicional (identificador de documento) durante la fase de entrenamiento. Este vector actúa como una "memoria distributiva global" que almacena el contexto semántico que falta en la ventana local, permitiendo capturar el sentido y la intención global del dictamen patológico del TCGA.

La evaluación de este modelo se estructuró mediante un flujo de trabajo iterativo diseñado para garantizar el determinismo y evitar el sesgo por fuga de datos:

1. **Estructuración mediante TaggedDocument:** A diferencia de los modelos previos, el algoritmo Doc2Vec requiere asociar explícitamente cada lista de tokens con una etiqueta única mediante la clase TaggedDocument. Esta etiqueta funciona en la arquitectura interna como un token especial que aprende la representación del documento en paralelo con los términos léxicos.
2. **Control de Hiperparámetros y Determinismo:** Se estableció un tamaño de vector de 100 dimensiones ("*vector_size=100*") para homogeneizar la comparativa. Se configuró un total de 20 épocas de entrenamiento ("*epochs=20*"), siguiendo el consenso científico de que los vectores de documento requieren un mayor número de pasadas sobre el corpus para estabilizarse. Asimismo, se fijó la semilla pseudo-aleatoria ("*seed=42*") para asegurar la reproducibilidad del espacio latente.
3. **Inferencia Dinámica Inédita:** Para evaluar el conjunto de prueba (*Test*), se recurrió al método "*infer_vector()*". Este procedimiento es crucial, ya que el modelo "congela" los pesos de las palabras previamente aprendidos y realiza una optimización por descenso de gradiente al vuelo para calcular de forma limpia la coordenada del informe inédito sin alterar la red.

Estrategia de preprocesamiento	Macro-F1 (Regresión Logística)
text_lemma	0.4692
text_basico	0.4505
text_stem_nltk	0.4422
text_pos	0.4310
text_lemma_nltk	0.4276
text	0.4138

- **El éxito de la Lematización** (text_lemma: 0.4692): Alcanzó el rendimiento óptimo. Reducir la variabilidad morfológica disminuye la dispersión matemática (*sparsity*), mientras que la conservación de los nexos mantiene intacto el flujo gramatical, permitiendo que la ventana de contexto de Doc2Vec opere con máxima eficiencia.
- **El colapso del Filtrado POS** (text_pos: 0.4310): Su bajo desempeño demuestra empíricamente la fuerte dependencia sintáctica del algoritmo. Al eliminar verbos y conectores, se destruye la secuencia natural de la frase, privando al modelo del contexto necesario para aprender prediciendo palabras adyacentes.
- **Degradación por Texto Crudo** (text: 0.4138): Obtuvo el peor resultado absoluto. El exceso de ruido (signos de puntuación, mayúsculas y códigos residuales) genera una alta cardinalidad que satura el espacio latente, impidiendo la correcta generalización del vector de documento.

En conclusión, los datos empíricos demuestran que para maximizar el rendimiento de representaciones a nivel de documento como Doc2Vec en el dominio oncológico, es imprescindible aplicar una normalización morfológica estricta (lematización) que reduzca el tamaño del vocabulario sin llegar a destruir la estructura sintáctica de la redacción clínica.

2.3.4. Empleo de embeddings contextuales

Tras evaluar el rendimiento de los embeddings estáticos, nos decantamos por el uso de embeddings contextuales. Estos, a diferencia de los embeddings estáticos, son capaces de diferenciar la implicación de la palabra en función del contexto en el que se emplea.

2.3.4.1. BERT

Dentro de los embeddings contextuales nos decantamos por el uso del modelo BERT. Este, se basa en Transformers y para su uso nos basamos principalmente en las siguientes razones:

- A diferencia de las redes recurrentes tradicionales, BERT analiza la secuencia de texto de manera simultánea, capturando así el contexto bidireccional de cada término.
- BERT incluye múltiples capas que calculan dinámicamente la relevancia de cada palabra respecto a las demás del informe. Esto permite que se les dé más importancia a las palabras con mayor relevancia en vez de a palabras más comunes con menor aporte para la clasificación sin necesidad de tanto preprocesamiento.
- En vez de tener que entrenar una red desde cero, BERT nos permite usar un modelo ya preentrenado. Para adaptarlo a nuestro proyecto, le aplicamos un proceso de Fine-Tuning, sobre la información que BERT extrae del texto, finalmente le conectamos una nueva capa neuronal con cuatro salidas correspondientes a los cuatro estadios posibles (T1, T2, T3, T4) la cual fue necesario entrenarla para que supiese clasificar los informes en sus estadios correspondientes.

Por otro lado, BERT cuenta con una limitación bastante importante, solo puede procesar un máximo de 512 tokens por documento. (añadir resolución de la limitación)

Otra limitación con la que nos encontramos es su alto coste computacional, por lo que tuvimos que recurrir a ejecutarlo en la GPU de Google Colab. Además, BERT es especialmente sensible a sobreajuste (“overfitting”) por lo que decidimos no implementar un aumento de datos.

Algunos de los parámetros más importantes de dicho modelo son los siguientes:

- “*truncation = true*”, “*padding=true*”:
Debido a que el máximo de tokens que es capaz de leer por documento es de 512, los textos que sobrepasaban esa longitud fueron truncados (“*truncation = true*”) priorizando la información más importante que típicamente encabeza los informes médicos. Por otro lado, aquellos informes que no llegan a dicha longitud fueron rellenados con 0 para homogeneizar los lotes matriciales (“*padding = true*”).
- “*num_train_epochs = 3*”:
Debido a que es un modelo ya preentrenado decidimos optar por el uso de 3 épocas para entrenar la nueva capa añadida, evitando también el sobreajuste a causa de un gran número de épocas.

- “*compute_metrics*”:

La API de Hugging Face evalúa los modelos en base al Accuracy por lo que se añadió una función de evaluación para forzar al modelo a optimizar el Macro-F1.

2.3.5. Empleo de modelos de Deep Learning

Tras evaluar el rendimiento de las representaciones vectoriales estáticas y los algoritmos de Machine Learning tradicional, esta investigación explora el uso de arquitecturas de Aprendizaje Profundo (*Deep Learning*). Si bien técnicas como FastText o TF-IDF han demostrado una notable solidez en la captación de similitudes semánticas y frecuencias sub-léxicas, estos enfoques presentan una limitación inherente: la pérdida parcial o total de la secuencialidad.

El lenguaje clínico empleado por los patólogos en los historiales médicos es rico en estructuras sintácticas complejas, negaciones y dependencias a largo plazo (por ejemplo, distinguir entre un "tumor primario con metástasis" y "ausencia de metástasis en el tumor"). Para capturar esta complejidad geométrica y secuencial del texto, se ha optado por implementar redes neuronales artificiales, las cuales no solo evalúan la presencia de terminología médica, sino también su disposición espacial y temporal dentro de la oración.

2.3.5.1. CNN (Convolutional Neural Network)

Este modelo, inicial y primordialmente es empleado para el procesamiento de imágenes (2D), pero puede ser adaptada a series temporales y texto demostrando que extrae eficazmente características locales.

Lo más destacable de este modelo es que a diferencia de otros como de Bag-Of-Words en los que ignoran el orden de las palabras, una red neuronal convolucional (CNN 1D) aplica filtros deslizantes por las diferentes secuencias textuales, lo que permite detectar patrones conservando el orden de lectura.

La estructura empleada para este caso particular viene dada de la siguiente manera:

- **Capa de Proyección (*Embedding Layer*):** Se implementó una matriz de *Word Embeddings* de 100 dimensiones **entrenable desde cero (*from scratch*)**. Aunque se experimentó aplicando *Fine-Tuning* (Ajuste Fino) sobre vectores pre-entrenados localmente con FastText, esta aproximación empeoró el rendimiento. La necesidad de vectorizar el texto en Keras obligó a truncar el vocabulario,

enmascarando términos clínicos infrecuentes bajo un token genérico (<OOV>) y anulando la principal ventaja sub-léxica de FastText. Por tanto, se priorizó la inicialización aleatoria, permitiendo a la red aprender mediante retropropagación una representación geométrica adaptada de forma exclusiva al léxico oncológico del corpus TCGA.

- **Extracción de Características (*Conv1D*):** Se configuró con **256 filtros** y una ventana amplia de lectura (**kernel_size=10**). De este modo, la red analiza el historial en bloques consecutivos de 10 palabras, buscando combinaciones diagnósticas relevantes.
- **Agrupamiento y Clasificación:** Una capa *GlobalMaxPooling1D* extrae la señal clínica más intensa de cada filtro (independientemente de su posición en el texto). Posteriormente, la información atraviesa una capa densa (128 neuronas) con una agresiva regularización *Dropout* (75%) para evitar el sobreajuste (*overfitting*), finalizando en una capa *Softmax* para predecir los estadios (T1-T4).

El modelo se entrenó durante 10 épocas optimizadas con *Adam*. Para contrarrestar el desbalanceo natural del corpus, se inyectó una matriz de pesos de clase (*class weights*) en la función de pérdida, obligando a la red a penalizar severamente los fallos en la detección de los estadios tumorales minoritarios.

Estrategia preprocesamiento	Macro-F1
text	0.713287
text_basico	0.624208
text_lemma	0.619489
text_lemma_nltk	0.616871
text_stem_nltk	0.612287
text_pos	0.608690

La evaluación cruzada frente a los distintos preprocesamientos reveló un fenómeno diferencial: el modelo alcanzó su máximo rendimiento predictivo (**Macro-F1 de 0.7148**) al ser alimentado con el texto crudo.

Este resultado subraya la naturaleza espacial de la CNN. Al emplear una ventana de lectura de 10 palabras, el algoritmo depende de la integridad de la estructura gramatical (incluyendo preposiciones y puntuación). Aplicar técnicas reductivas agresivas, como el *Stemming* o el filtrado POS, destruyó la coherencia de la secuencia léxica, introduciendo ruido en el campo receptivo de los filtros convolucionales y provocando caídas de hasta 13 puntos porcentuales en la métrica F1 respecto al texto íntegro.

2.3.5.2. RNN (Recurrent Neural Network)

Aunque la arquitectura convolucional (CNN) extrae eficazmente patrones sintácticos locales, el lenguaje clínico presenta el desafío de las **dependencias a larga distancia** (por ejemplo, una negación separada del diagnóstico principal por decenas de palabras). Para capturar esta secuencialidad global, se exploró la implementación de una Red Neuronal Recurrente con celdas de Memoria a Corto y Largo Plazo (**LSTM**).

A diferencia de las CNN, que leen mediante ventanas de tamaño fijo, las LSTM procesan el texto palabra por palabra. Mediante un mecanismo interno de "estado de celda" regulado por compuertas matemáticas, la red decide de forma autónoma qué terminología médica debe retener en memoria a medida que avanza la lectura y qué información irrelevante debe descartar. Esto permite a la arquitectura modelar el historial clínico oncológico no como fragmentos aislados, sino como un continuo narrativo.

2.3.5.2.1. LSTM

Para abordar las dependencias sintácticas a larga distancia presentes en los historiales clínicos, se implementó una arquitectura recurrente basada en celdas de Memoria a Corto y Largo Plazo (**LSTM**). A diferencia de las redes convolucionales (CNN) que operan mediante ventanas de lectura estáticas, la topología LSTM procesa el texto de forma estrictamente secuencial. Gracias a su mecanismo interno de compuertas matemáticas (*forget*, *input* y *output gates*), la red arrastra el contexto acumulado palabra por palabra, decidiendo de forma autónoma qué terminología médica debe retener en su estado de celda y qué información irrelevante debe descartar.

- **Capa de Proyección (*Embedding*):** Matriz entrenable desde cero que proyecta el vocabulario de 5.000 términos en un espacio continuo de 100 dimensiones.
- **Capa Recurrente (*LSTM*):** El núcleo extractor del modelo, configurado con **128 unidades ocultas**. Para mitigar el sobreajuste durante la lectura secuencial, se aplicó una regularización interna desconectando aleatoriamente el 20% de las conexiones de entrada ($\text{dropout}=0.2$) y el 20% de las conexiones recurrentes del estado oculto ($\text{recurrent_dropout}=0.2$).
- **Capas Densas y Salida:** Una capa completamente conectada de 64 neuronas (activación *ReLU*) seguida de un *Dropout* severo del 50%. Finalmente, una capa *Softmax* de 4 neuronas genera la distribución de probabilidad para los cuatro estadios tumorales.

Dado el fuerte desequilibrio natural en la distribución de los estadios del cáncer (clases minoritarias como T4 frente a mayoritarias como T1), no se recurrió al aumento físico de datos, sino a una penalización matemática. Se calculó una matriz de pesos de clase (*class weights*) balanceada y se inyectó directamente en la función de pérdida (*Sparse Categorical Crossentropy*). Esto obligó al optimizador *Adam* a penalizar con mayor severidad los errores cometidos sobre las clases minoritarias, garantizando que la red no ignorara los tumores más severos.

El entrenamiento se ejecutó de forma sistemática sobre las seis variantes de preprocesamiento del proyecto, procesando lotes (*batches*) de 32 historiales durante 10 épocas para cada configuración. Para cada uno de estos experimentos, se reservó un 10% del conjunto de entrenamiento estricto como datos de validación cruzada (`validation_split=0.1`) para monitorizar la capacidad de generalización del modelo en cada iteración.

Estrategia preprocesamiento	Macro-F1
text	0.330396
text_stem_nltk	0.281685
text_lemma_nltk	0.279180
text_basico	0.263505
text_lemma	0.260758
text_pos	0.253407

Al evaluar la arquitectura LSTM unidireccional frente a las distintas estrategias de limpieza de texto, los resultados evidenciaron un rendimiento general más discreto que el modelo convolucional, pero con una tendencia lingüística idéntica:

- El modelo alcanzó su máximo rendimiento al ser alimentado con el **texto crudo (text)**, logrando un **Macro-F1 de 0.3303**.
- Las variantes sometidas a reducciones morfológicas intermedias experimentaron caídas notables, como el *Stemming* (`text_stem_nltk`: 0.2816) y la Lematización (`text_lemma_nltk`: 0.2791).
- Las representaciones más agresivas, como el filtrado de categorías gramaticales (`text_pos`: 0.2534), obtuvieron los peores desempeños.

Esta métrica confirma la gran vulnerabilidad de las redes recurrentes ante la alteración sintáctica. Dado que la LSTM procesa la información de forma estrictamente secuencial, eliminar conectores, signos de puntuación o truncar el vocabulario destruye el "hilo conductor" de la narrativa clínica, impidiendo que la memoria interna de la red retenga las dependencias lógicas a largo plazo necesarias para una correcta clasificación del estadio tumoral.

2.3.5.2.2. Bi-LSTM

Para maximizar la extracción de contexto global de los historiales clínicos, se evolucionó la arquitectura recurrente hacia una topología **Bidireccional (Bi-LSTM)**. A diferencia de la LSTM estándar, que lee el texto exclusivamente en un sentido (de inicio a fin), la Bi-LSTM procesa la secuencia léxica en ambas direcciones de forma simultánea. Esta aproximación es fundamental en la narrativa oncológica, donde el diagnóstico central a menudo depende tanto de los modificadores que le preceden (por ejemplo, una negación) como de las observaciones que aparecen al final del documento, permitiendo a la red construir una representación semántica completa de la sintaxis del patólogo.

Para este bloque experimental, se incrementó notablemente la capacidad receptiva del modelo. Se amplió el diccionario del tokenizador a 10.000 palabras (*"VOCAB_SIZE"*) y se extendió la ventana de lectura a los primeros 512 tokens (*MAX_LEN*) de cada historial, garantizando la captura de informes médicos inusualmente largos. La red se estructuró de la siguiente manera:

- **Capa de Proyección:** Una matriz de *Embeddings* más profunda, proyectando el vocabulario en un espacio latente de 128 dimensiones.
- **Extracción Bidireccional:** El núcleo del modelo consiste en una envoltura bidireccional sobre una capa LSTM de 64 unidades. Al evaluar el texto hacia adelante y hacia atrás, y concatenar sus estados ocultos finales (*"return_sequences=False"*), el modelo sintetiza el contexto global en un único vector de características.
- **Clasificación y Regularización:** El vector resultante se somete a una agresiva regularización *Dropout* (50%) para evitar el sobreajuste. Posteriormente, atraviesa una capa densa intermedia de 64 neuronas con activación *ReLU*, finalizando en una capa *Softmax* para emitir la probabilidad sobre los cuatro estadios (T1-T4).

Estrategia preprocesamiento	Macro-F1
text_pos	0.5290
text_lemma	0.5278
text_stem_nltk	0.5125
text_basico	0.5112
text	0.4956
text_lemma_nltk	0.4926

La evaluación de la arquitectura Bi-LSTM arrojó una mejora sustancial respecto a su variante unidireccional (subiendo de un F1 de ~0.33 a ~0.52), demostrando que la lectura en ambos sentidos es vital para comprender el lenguaje oncológico. No obstante, el hallazgo más relevante de este experimento fue la inversión total de la tendencia frente al preprocesamiento.

Los resultados obtenidos fueron los siguientes:

- El mejor desempeño se logró aplicando un filtrado estricto por categorías gramaticales (**text_pos: 0.5290**), seguido muy de cerca por la lematización pura (**text_lemma: 0.5278**).
- Por el contrario, el **texto crudo (text: 0.4956)** y la lematización con NLTK (**text_lemma_nltk: 0.4926**) arrojaron las peores métricas del bloque.

Discusión del fenómeno: Este cambio de paradigma se explica por la combinación de la bidireccionalidad y la extensa ventana de lectura configurada (*"MAX_LEN = 512"*). Al procesar secuencias tan largas en dos direcciones, el texto crudo introduce un exceso de "ruido sintáctico" (preposiciones, determinantes, conectores y redundancias) que termina saturando la memoria interna de las celdas LSTM, diluyendo la señal de las palabras verdaderamente críticas.

Al aplicar técnicas reductivas como el filtrado POS o la lematización, el historial clínico se "comprime". Se eliminan los tokens vacíos de significado y la secuencia pasa a ser una sucesión densa de conceptos médicos puros. Al ser más corta y rica en información, la Bi-LSTM logra conectar conceptos clínicos distantes (como un diagnóstico inicial y una negación de metástasis al final del documento) de forma mucho más eficiente y sin pérdida de gradiente, superando así al texto sin procesar.

2.3.5.3. Aumentado de datos.

El análisis inicial de los datos reveló un desequilibrio de clases significativo. La distribución de la variable objetivo presentaba una gran sobrerepresentación de la clase T2 mientras que había una escasez en cuanto a la clase T4.

Este desbalance representa un gran problema sobre todo en las arquitecturas de aprendizaje automático como las Redes Neuronales Convolucionales (CNN), redes recurrentes de Memoria a Corto-Largo Plazo (LSTM) las redes Bidireccionales (Bi-LSTM). Si este desbalance no fuese tratado causaría una tendencia clara a la predicción de la clase T2.

Para solucionar este desbalance, se integró una técnica de aumentado de datos sintético mediante sobremuestreo de la clase minoritaria (*Oversampling*). Esta técnica se aplicó sobre las secuencias numéricas de texto ya codificadas y con el relleno aplicado. Cabe recalcar que solo se aplicó en el conjunto de entrenamiento.

La introducción de muestras duplicadas mediante sobremuestreo conlleva al riesgo de sobreajuste ya que la red neuronal memoriza los patrones de la clase T4 en lugar de generalizarlos. Por ello, se diseñaron dos mecanismos integrados en la arquitectura de los modelos:

1. Regularización por *Dropout*: Se forzó a la red a no depender de características locales específicas mediante la desactivación aleatoria de neuronas durante el entrenamiento.
2. Monitorización *Early Stopping*: Se integró un mecanismo donde si el modelo comienza a memorizar los datos sobremuestreados (reflejado en un aumento de la función de pérdida de validación), el entrenamiento se detenía para restaurar los pesos óptimos.

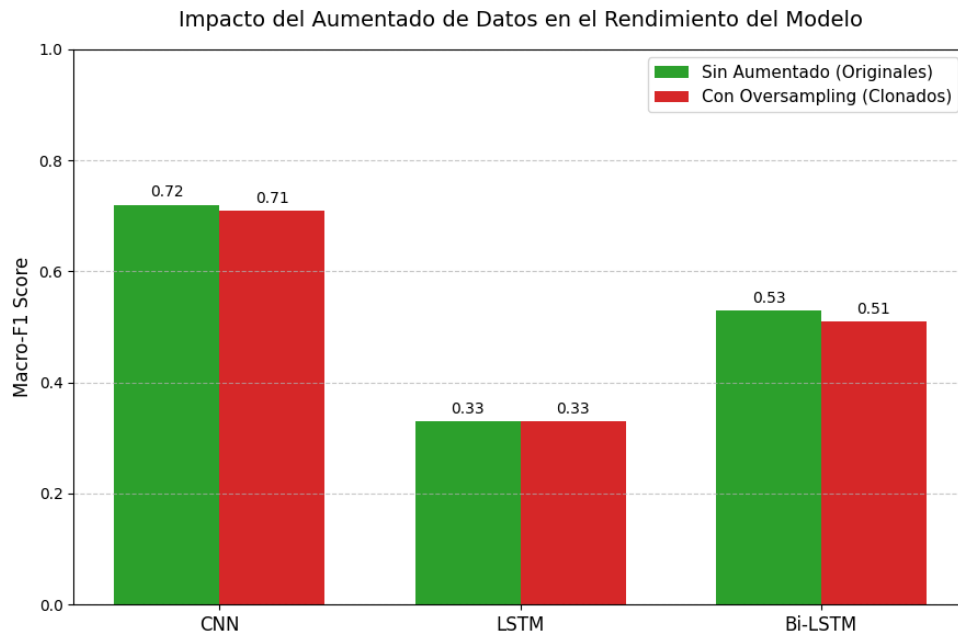


Figura 12

Análisis del impacto del aumentado de datos:

A pesar del sobremuestreo, su aplicación en nuestras redes no supuso una mejora notable sobre la métrica Macro-F1 (*Figura 12*).

Tras analizar los resultados, se determinó que esto se debe a dos motivos principales:

1. Sobreajuste por memorización: Las redes terminaron memorizando los pacientes concretos del estadio T4 en lugar de aprender y generalizar.
2. Fracaso respecto a test: Cuando nuestro modelo se enfrenta a un conjunto de datos que nunca ha visto, la memorización obtenida anteriormente hace que no le sirva para clasificar por lo que la métrica Macro-f1 no mejora.

Conclusión:

Para solucionar el desbalanceo original evitando la memorización, la alternativa más robusta ha sido el uso de Pesos de Clase (Class Weights). Esta técnica modifica la función de coste castigando más a los errores de la clase minoritaria (T4), pero manteniendo intactos los datos originales.

3. Conclusiones

Tras completar la experimentación con diversas técnicas de Procesamiento de Lenguaje Natural aplicadas a la clasificación de estadios tumorales (T1-T4) en el corpus médico TCGA, hemos extraído una serie de conclusiones fundamentales sobre el comportamiento de los modelos de Machine Learning y Deep Learning frente al lenguaje clínico:

- **El dilema del Preprocesamiento (Texto crudo vs. Texto limpio):** A lo largo del proyecto hemos comprobado empíricamente que no existe un "preprocesamiento perfecto", sino que depende estrictamente de la arquitectura empleada. Modelos espaciales (como la CNN) o de frecuencias (como la Regresión Logística y SVM) obtuvieron sus mejores resultados (Macro-F1 > 0.70) utilizando el **texto crudo** (text). Esto demuestra que los números, tamaños en centímetros y códigos médicos originales son vitales para clasificar un tumor. Sin embargo, para arquitecturas que necesitan leer secuencias muy largas y comprimir contexto (como Doc2Vec o Bi-LSTM), el texto crudo actúa como ruido. En estos casos, aplicar reducciones agresivas como la **lematización** o el **filtrado POS** fue la única forma de maximizar su rendimiento.
- **La sorpresa de los Modelos Clásicos:** A menudo se asume que el Aprendizaje Profundo siempre supera a los algoritmos tradicionales. Sin embargo, nuestra investigación demuestra que un modelo lineal simple como la **Regresión Logística con Pesado Binario** (0.7040) puede competir e incluso superar a arquitecturas complejas como LSTM o Doc2Vec. En dominios con vocabularios técnicos muy específicos, la simple confirmación de que una palabra clave existe (1) o no existe (0) en el historial es un predictor extremadamente potente.
- **El reto del desbalanceo clínico y el peligro del Oversampling:** Al tratar con clases minoritarias tan críticas como el estadio T4, descubrimos que generar datos sintéticos (Oversampling) en redes neuronales provocaba un efecto indeseado de sobreajuste y aumentaba los falsos positivos (el modelo memorizaba los historiales de T4). La solución más robusta y segura desde un punto de vista clínico fue aplicar penalizaciones matemáticas mediante **Pesos de Clase** ("*class_weights*"), obligando al algoritmo a prestar atención a los casos graves sin alterar la distribución real de los datos originales.
- **Embeddings Locales frente a Modelos Generales:** El lenguaje médico es un dominio en sí mismo. Durante la evaluación de los *Embeddings*, confirmamos que modelos sub-léxicos entrenados desde cero con nuestros propios datos (como **FastText**) gestionan mucho mejor la variabilidad y las erratas del texto clínico que modelos pre-entrenados con corpus mundiales (como **GloVe**), los cuales fallan al no reconocer raíces médicas o términos truncados por el *Stemming*.

- **El ganador del estudio (CNN):** A nivel global, la **Red Neuronal Convolutiva (CNN)** demostró ser la arquitectura más sólida para este problema específico, logrando el Macro-F1 más alto (0.7132). Sus filtros espaciales resultaron ideales para detectar patrones locales y agrupaciones de palabras patológicas (n-gramas) directamente sobre el texto sin procesar, equilibrando la eficacia computacional con un alto poder predictivo.

En resumen, este proyecto evidencia que, en el Procesamiento de Lenguaje Natural aplicado a la medicina, la limpieza del texto y la selección del modelo deben estar completamente alineadas. Preservar la información clínica numérica y gestionar el desbalanceo de clases mediante penalizaciones han resultado ser las claves para convertir historiales de patología en datos estructurados y predecibles.

4. Bibliografía y Referencias

Artículos Científicos de referencia:

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient estimation of word representations in vector space*. arXiv preprint arXiv:1301.3781. (Base teórica para Word2Vec).

Documentación Oficial y Herramientas (**Webs**):

- **Gensim**: Documentación oficial para la implementación de Word2Vec, FastText y Doc2Vec en Python. Recuperado de: <https://radimrehurek.com/gensim/>
- **Hugging Face**: Documentación oficial de la librería Transformers (Uso del modelo BERT pre-entrenado). Recuperado de: <https://huggingface.co/docs/transformers/>
- **Keras & TensorFlow**: Guías oficiales de la API para la construcción de redes neuronales (CNN, LSTM y Bi-LSTM). Recuperado de: <https://keras.io/>
- **NLTK (Natural Language Toolkit)**: Documentación oficial para tokenización, eliminación de *stop words* y *stemming* (PorterStemmer). Recuperado de: <https://www.nltk.org/>
- **Scikit-Learn**: Guías de usuario y documentación de algoritmos de Machine Learning clásico (Logistic Regression, MultinomialNB, LinearSVC) y extracción de características (TF-IDF, CountVectorizer). Recuperado de: <https://scikit-learn.org/>
- **spaCy**: Documentación oficial sobre procesamiento de lenguaje natural de grado industrial (Lematización y filtrado POS). Recuperado de: <https://spacy.io/>
- **The Cancer Genome Atlas (TCGA)**: Portal oficial de datos genómicos y clínicos del Instituto Nacional del Cáncer. Recuperado de: <https://portal.gdc.cancer.gov/>